

Torus 网格生成模块：torus.rs 实现详解

2026 年 8 月 11 日

目录

1	概述	2
2	数学基础	2
2.1	Torus 的参数方程	2
2.2	几何直观	3
2.3	解析法向量	3
3	核心函数实现	3
3.1	torus_position: 标准坐标系下的位置计算	3
3.2	torus_position_frame: 任意坐标系下的位置计算	4
3.3	unfold_position: 平面展开坐标	5
3.4	generate_unfolded_quad_mesh: 展开四边形网格生成	5
3.5	四边形索引顺序	6
4	数据结构	6
4.1	输出三元组	6
4.2	下游构造	6
5	测试验证	7
5.1	test_torus_periodicity	7
5.2	test_torus_geometry	7
5.3	test_unfold_matches_3d	7
6	与其他模块的关系	8
7	总结	8

1 概述

`torus.rs` 模块实现了 Torus（环面/甜甜圈面）的参数化网格生成功能。该模块提供从数学参数方程计算 Torus 表面任意点三维坐标的核心工具，是当前 Torus 切割项目的基础几何生成器。

模块对外提供的真实接口（其余函数为内部实现细节）：

- `torus_position(u, v, major_r, minor_r)`：标准轴对齐坐标系下，由参数 (u, v) 计算 Torus 表面三维坐标。
- `torus_position_frame(u, v, major_r, minor_r, center, axis, u_axis, v_axis)`：在任意坐标系（中心/轴线/两套面内基轴）下计算三维坐标，与 `SurfaceModel::Torus` 及 `knot` 解析曲线共用同一坐标框架。
- `unfold_position(u, v, major_r, minor_r)`：将 (u, v) 展开为平面直角坐标（UV 平面视图）。
- `generate_unfolded_quad_mesh(major_r, minor_r, res_u, res_v)`：直接由参数方程生成展开的四边形网格（顶点 + UV + 四边形索引），不依赖已有网格，也不生成跨越接缝的回绕面片。

注意：本模块不提供 `torus_normal`、`generate_torus_mesh`、`generate_torus_mesh_vertices` 等函数；法向量在渲染阶段由半边网格现算（平面法或顶点法线），而不是在几何生成阶段预存。

2 数学基础

2.1 Torus 的参数方程

Torus 由两个参数定义：

- 大半径 R （major radius）：从 Torus 中心到管道中心的距离
- 小半径 r （minor radius）：管道的半径

Torus 的标准参数方程为：

$$\begin{aligned}x(u, v) &= (R + r \cos v) \cos u \\y(u, v) &= (R + r \cos v) \sin u \\z(u, v) &= r \sin v\end{aligned}\tag{1}$$

其中：

- $u \in [0, 2\pi)$: 绕大圆的角度参数（环向角）
- $v \in [0, 2\pi)$: 绕管道的角度参数（管向角）

轴线为 z 轴，管道截面始终位于 xy 平面内以 $(R \cos u, R \sin u, 0)$ 为圆心的圆上。

2.2 几何直观

可以将 Torus 的形成过程理解为：

1. 在 xy 平面上，以 $(R, 0, 0)$ 为圆心、 r 为半径画一个圆（管道截面）
2. 将这个圆绕 z 轴旋转一周（ u 从 0 到 2π ）

参数 v 控制管道截面上的位置，参数 u 控制管道绕 z 轴的旋转角度。

2.3 解析法向量

Torus 表面上点 (u, v) 处的外法向量由偏导叉积给出（详见 `surface.rs` 的几何推导，此处仅给出结论）：

$$\mathbf{n}(u, v) \propto (\cos v \cos u, \cos v \sin u, \sin v) \quad (2)$$

该向量指向 Torus 外侧，仅与角度参数有关、与半径无关。实际渲染时统一在 `app.rs` 的 `rebuild_render_state` 中按面或按顶点计算，不在 `torus.rs` 内单独实现。

3 核心函数实现

3.1 torus_position: 标准坐标系下的位置计算

```

1 pub fn torus_position(u: f64, v: f64, major_r: f64, minor_r: f64) ->
  Vec3 {
2     let cu = u.cos();
3     let su = u.sin();
4     let cv = v.cos();
5     let sv = v.sin();
6     let rr = major_r + minor_r * cv;
7     Vec3::new(
8         rr as f32 * cu as f32,
9         rr as f32 * su as f32,
10        minor_r as f32 * sv as f32,
11    )
12 }
```

实现细节:

- 使用三角函数的局部变量 ($\cos u$, $\sin u$, $\cos v$, $\sin v$) 避免重复计算
- 中间变量 $rr = R + r \cos v$ 表示当前管道截面圆心到 z 轴的距离
- 计算在 f64 精度下进行, 最后转换为 f32 以匹配 GPU 渲染需求

数学对应: 直接实现公式 (1), 与轴线为 z 轴、面内基轴为 $(1, 0, 0)$ 与 $(0, 1, 0)$ 的标准 `SurfaceModel::torus_from_params` 一致。

3.2 torus_position_frame: 任意坐标系下的位置计算

```
1 pub fn torus_position_frame(  
2     u: f64, v: f64, major_r: f64, minor_r: f64,  
3     center: Vec3, axis: Vec3, u_axis: Vec3, v_axis: Vec3,  
4 ) -> Vec3 {  
5     let cu = u.cos();  
6     let su = u.sin();  
7     let cv = v.cos();  
8     let sv = v.sin();  
9     let rr = major_r + minor_r * cv;  
10    let x = rr * cu;  
11    let y = rr * su;  
12    let z = minor_r * sv;  
13    center + u_axis * x as f32 + v_axis * y as f32 + axis * z as f32  
14 }
```

设计动机: 当导入 OBJ 网格并拟合出任意姿态的 Torus (中心 `center`、轴线 `axis`、面内基轴 `u_axis/v_axis`) 时, 由参数方程生成的网格顶点必须与 knot 解析曲线 (`SurfaceModel::generate_torus_knot_line`) 落在同一个三维坐标框架内, 否则切割线与网格会错位。torus_position_frame 与 generate_torus_knot_line 共用相同的 center/axis/u_axis/v_axis 映射, 保证两者坐标一致。

3.3 unfold_position: 平面展开坐标

```
1 /// 将环面体的极坐标 (u, v) 展开为平面直角坐标。  
2 /// x = u * R (沿大圆的弧长), y = v * r (沿小圆的弧长), z = 0  
3 pub fn unfold_position(u: f64, v: f64, major_r: f64, minor_r: f64) ->  
4     Vec3 {  
5     Vec3::new((u * major_r) as f32, (v * minor_r) as f32, 0.0)  
6 }
```

用途：UV 平面视图 (`ViewMode::Unfolded`) 下，把每个顶点的 (u, v) 参数直接映射为平面直角坐标 $(x, y, z) = (uR, vr, 0)$ ，用于在平面上查看/编辑切割结果。在 `app.rs` 的 `to_view_mesh / rebuild_render_state` 中被调用。

3.4 `generate_unfolded_quad_mesh`: 展开四边形网格生成

```
1 pub fn generate_unfolded_quad_mesh(  
2     major_r: f64, minor_r: f64, res_u: usize, res_v: usize,  
3 ) -> (Vec<Vec3>, Vec<Vec2>, Vec<QuadIndex>) {  
4     // res_u+1 x res_v+1 顶点: 同时包含 u=0 与 u=2 (v 同理)  
5     // 这两组顶点在 UV 上是独立的, 但 3D 坐标相同。  
6     let nu = res_u + 1;  
7     let nv = res_v + 1;  
8     let mut positions = Vec::with_capacity(nu * nv);  
9     let mut uvs = Vec::with_capacity(nu * nv);  
10    for i in 0..nu {  
11        let u = 2.0 * PI * i as f64 / res_u as f64;  
12        for j in 0..nv {  
13            let v = 2.0 * PI * j as f64 / res_v as f64;  
14            let pos = torus_position(u, v, major_r, minor_r);  
15            positions.push(pos);  
16            uvs.push(Vec2::new(u as f32, v as f32));  
17        }  
18    }  
19    // 四边形: 绝不回绕——只连接相邻列与相邻行  
20    let mut quads = Vec::with_capacity(res_u * res_v);  
21    for i in 0..res_u {  
22        for j in 0..res_v {  
23            let v00 = i * nv + j;  
24            let v10 = (i + 1) * nv + j;  
25            let v11 = (i + 1) * nv + (j + 1);  
26            let v01 = i * nv + (j + 1);  
27            quads.push((v00, v10, v11, v01));  
28        }  
29    }  
30    (positions, uvs, quads)  
31 }
```

关键事实 (纠正旧文档):

- 顶点总数为 $(res_u + 1) \times (res_v + 1)$ ，不是 $res_u \times res_v$ 。

- 这是因为网格同时包含参数域两端 $u = 0$ 与 $u = 2\pi$ （以及 $v = 0$ 与 $v = 2\pi$ ）的顶点：它们在 UV 空间是独立顶点（便于展开平面视图的左/右、上/下边界各自独立），但对应同一个 3D 位置。
- 四边形总数为 $res_u \times res_v$ 。
- 索引计算：顶点 (i, j) 的线性索引为 $i \times (res_v + 1) + j$ 。
- 不生成跨越接缝的四边形：循环仅遍历 $i \in [0, res_u)$ 、 $j \in [0, res_v)$ ，因此不会出现 $u = 2\pi \rightarrow 0$ 或 $v = 2\pi \rightarrow 0$ 的回绕面片；接缝处由两端各自的独立顶点承载。

3.5 四边形索引顺序

每个参数空间四边形 $\{(i, j), (i + 1, j), (i + 1, j + 1), (i, j + 1)\}$ 的顶点按

$$(v_{00}, v_{10}, v_{11}, v_{01}) \quad (3)$$

排列，其中 $v_{00} = i \cdot nv + j$ 、 $v_{10} = (i + 1) \cdot nv + j$ 、 $v_{11} = (i + 1) \cdot nv + (j + 1)$ 、 $v_{01} = i \cdot nv + (j + 1)$ 。该顺序保持逆时针绕向，配合 `HalfEdgeMesh::from_quads` 构造流形网格。

4 数据结构

4.1 输出三元组

`generate_unfolded_quad_mesh` 返回：

分量	类型	含义
<code>positions</code>	<code>Vec<Vec3></code>	三维坐标，长度 $(res_u + 1)(res_v + 1)$
<code>uvs</code>	<code>Vec<Vec2></code>	参数坐标 (u, v) ，与 <code>positions</code> 一一对应
<code>quads</code>	<code>Vec<QuadIndex></code>	四边形顶点索引四元组，长度 $res_u \cdot res_v$

表 1: `generate_unfolded_quad_mesh` 输出

4.2 下游构造

- Quad 模式（默认）：`HalfEdgeMesh::from_quads(&positions, &uvs, &quads)` 直接构造半边网格（见 `half_edge.rs`）。
- Delaunay 模式：由 `delaunay::generate_delaunay_mesh` 生成三角形，再用 `HalfEdgeMesh::from(&uvs, &triangles)` 构造。
- OBJ 导入：由 `obj_loader::load_obj_as_half_edge` 直接得到半边网格。

5 测试验证

模块包含三个单元测试(纠正旧文档:不存在 `test_torus_vertex_count` / `test_torus_position`)

5.1 test_torus_periodicity

验证环面 u/v 均以 2π 为周期:

```
1 #[test]
2 fn test_torus_periodicity() {
3     let (major, minor) = (2.0, 0.5);
4     let (u, v) = (1.2, 2.4);
5     let p = torus_position(u, v, major, minor);
6     assert!(torus_position(u + 2.0*PI, v, major, minor).distance(p) <
7         1e-5);
8     assert!(torus_position(u, v + 2.0*PI, major, minor).distance(p) <
9         1e-5);
10 }
```

即 $\text{torus_position}(u+2\pi, v) == \text{torus_position}(u, v)$

5.2 test_torus_geometry

验证几何性质: $v = 0$ 时点在最外侧 (距中心 $= R + r$); $u = 0, v = \pi/2$ 时管顶在 $+z$ ($z = r$, 主圆半径 $= R$); $v = \pi$ 时在内圈 (距中心 $= R - r$)。

5.3 test_unfold_matches_3d

验证 `unfold_position` 与 `torus_position` 共用同一 (u, v) 参数域: 展开坐标 $x = uR, y = vr$ 与输入一致, 且 3D 位置有限、可计算。

6 与其他模块的关系

`torus.rs` 是整个切割流程的几何起点:

1. 生成初始 Torus 网格 (Quad / Delaunay / OBJ 三源之一)
2. 转换为半边数据结构进行拓扑管理
3. 切割算法基于参数坐标 (u, v) 执行
4. 最终结果渲染并导出

```

torus.rs
|
+-- generate_unfolded_quad_mesh()  --(positions, uvs, quads)-->
|      (Quad 模式)
|
+-- delaunay::generate_delaunay_mesh()  --(positions, uvs, tris)-- (Delaunay 模式)
|
+-- obj_loader::load_obj_as_half_edge()                                (OBJ 模式)
|
v
HalfEdgeMesh (from_quads / from_triangles)
|
v
cut.rs (网格切割: cut_mesh_by_grid / cut_mesh_by_knots)
|
v
color_scheme.rs (补片着色) + render/ (wgpu 渲染) + export/ (OBJ/STL/PLY 导出)

```

图 1: torus.rs 在项目中的位置

7 总结

`torus.rs` 模块提供了 `Torus` 网格生成的完整实现，具有以下特点：

1. 数学正确性：严格遵循 `Torus` 参数方程；`torus_position_frame` 支持任意姿态以适配 OBJ 拟合结果。
2. 展开网格的边界处理：顶点数取 $(res_u + 1) \times (res_v + 1)$ ，显式保留接缝两端顶点，且不生成回绕四边形，保证 UV 平面视图与 3D 视图一致。
3. 参数化支持：保留 (u, v) 坐标，为后续切割与着色提供基础。
4. 可测试性：包含周期性、几何正确性、展开映射三类单元测试。

该模块是整个 `Torus` 切割项目的几何基础，其正确性和效率直接影响后续的切割和渲染效果。